UNIVERSITY TECHNICAL WALKTHROUGH

Fitness Explorer Backend

Sequelize ORM · Express · MariaDB

A beginner-friendly explanation of the actual project structure, database mapping, API operations, image upload flow, frontend integration, verification, and presentation.

Prepared: 8 November 2026

Database: fitness_explorer (schema and six-row count supplied by the student)

Data safety: Read-only inspection; no database connection, migration, or schema synchronization was performed for this report.

Verification boundary: Manual API results are clearly attributed to the student's supplied report. Local lint, build, and frontend tests are separately reported.

```
PROJECT ROOT
C:\Users\kerob\Downloads\fitness-explorer-advanced-dashboard\fitness-explorer
```

CONTENTS

Table of contents

1. Project overview and inspection basis

2. How the backend was developed

3. Backend folder structure and file guide

4. Dependencies and environment configuration

5. Database table and Exercise model

6. Server startup, middleware, and errors

7. Routes, controllers, and model operations

8. Complete request and response flow

9. API endpoint reference

10. CRUD explained with Sequelize

11. Search, filtering, sorting, and pagination

12. Image upload from browser to disk and database

13. React, API client, and Zustand integration

14. PowerShell and curl.exe practice commands

15. Testing and verified results

16. Security, limitations, and future improvements

17. Supervisor presentation walkthrough

18. Supervisor questions and answers

19. Student-level conclusion

Appendix material is integrated into the relevant chapters.

CHAPTER 01

Project overview and inspection basis

> **Scope and data safety.** This walkthrough describes the source currently in the project and the database facts supplied by the student. The six-row count and table definition were not independently queried in this document-generation task. No database connection, migration, schema synchronization, or CRUD mutation was run while preparing this PDF.

Project snapshot

Item	What this report records
Project	Fitness Explorer: React exercise and workout application with a Node.js/Express backend.
Project root	`C:\Users\kerob\Downloads\fitness-explorer-advanced-dashboard\fitness-explorer`
Backend root	`C:\Users\kerob\Downloads\fitness-explorer-advanced-dashboard\fitness-explorer\backend`
Database	`fitness_explorer`, MariaDB/MySQL-compatible; six existing records as confirmed by the student.
Backend data layer	Sequelize 6.37.8; MySQL dialect; mysql2 3.24.6 driver.
Report date	8 November 2026

Inspection basis

The report was based on the current backend source, backend and root package manifests/lockfiles, frontend service and upload integration, root README and environment example, existing test files, and the backend learning guide. Environment files were inspected only for variable names; their values are not reproduced.

User-reported manual API results are presented separately from commands and checks run while generating this report. Examples use clearly identified test records and are not represented as database results from this report run.

Important current design point

The backend is not a raw-SQL application. CRUD and upload database access use Sequelize model operations. Sequelize translates model calls into database commands through the mysql2 driver; the application source does not manually issue SQL statements.

CHAPTER 02

How the backend was developed

From frontend to full-stack application

Fitness Explorer began as a React application. It contains exercise discovery, detail pages, favorites, workout building and tracking, a calendar, and progress views. Zustand is used for client state such as favorites, recently viewed exercises, and workout state.

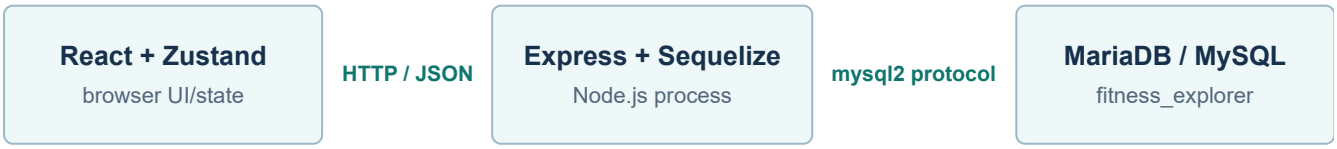
A backend adds a shared place to store and retrieve exercise records and uploaded images. The backend accepts HTTP requests from the React service layer, applies validation and business handling, communicates with MariaDB through Sequelize, and returns JSON. The browser does not connect directly to the database.

Why these technologies fit together

Technology	Role in this project
Node.js	Runs the JavaScript backend process and its server-side modules.
Express.js	Creates the HTTP server, middleware pipeline, API routes, and response handlers.
MariaDB / MySQL	Stores exercise rows in the existing fitness_explorer database.
Sequelize ORM v6	Maps the exercises table to a JavaScript model and performs database operations through methods.
mysql2	Database protocol driver used by Sequelize's MySQL dialect to communicate with MariaDB/MySQL.
dotenv	Loads backend configuration values from the backend environment file.
Multer	Parses a multipart file upload and writes it to disk.
Helmet	Adds standard security-related HTTP headers.
CORS	Allows the configured frontend browser origin to read API responses.
Zustand	Keeps frontend state; it is not the backend database or an ORM.

How XAMPP fits

The project configuration targets a MySQL-compatible service at a host and port, with defaults of localhost and 3306. If XAMPP is used to run MariaDB locally, XAMPP supplies that database service. Node.js runs the Express process separately. XAMPP is not imported by the JavaScript application; the code communicates with the database protocol endpoint configured by DB_HOST and DB_PORT.



Sequelize is the ORM, mysql2 is the low-level driver, MariaDB is the database server, and Express is the HTTP API server. Each has a separate responsibility.

CHAPTER 03

Backend folder structure and file guide

```
backend/
|-- .env                        # server/database settings (secret values omitted)
|-- package.json                # scripts and direct dependencies
|-- package-lock.json           # exact resolved package versions
|-- BACKEND_LEARNING_GUIDE.md   # existing learning notes
|-- uploads/                    # .gitkeep plus current image files
|-- src/
    |-- server.js               # Express setup and startup
    |-- config/
    |   |-- db.js               # shared Sequelize instance/authentication
    |-- routes/
    |   |-- exerciseRoutes.js   # URL/method to controller mapping
    |-- controllers/
    |   |-- exerciseController.js # HTTP validation and CRUD responses
    |   |-- uploadController.js   # image-to-exercise association
    |-- models/
    |   |-- Exercise.js         # database table definition
    |   |-- exerciseModel.js    # reusable Sequelize operations
    |-- middleware/
        |-- uploadMiddleware.js  # Multer file storage/filter/limits
```

File	Responsibility and connections
backend/src/server.js	Imports the route module and DB authentication function; registers Helmet, CORS, JSON parsing, static uploads, API routes, health handler, and error handler.
backend/src/config/db.js	Loads dotenv settings, creates one Sequelize instance, exports it and testDatabaseConnection().
backend/src/models/Exercise.js	Defines fields, table name, timestamps, unique rule, and declared indexes. It does not create the table.
backend/src/models/exerciseModel.js	Imports Exercise and Sequelize Op; builds filters and provides list/count/read/create/update/delete functions.
backend/src/routes/exerciseRoutes.js	Imports exercise and upload controllers plus Multer middleware; declares the exercise routes.
backend/src/controllers/exerciseController.js	Receives Express req/res, validates IDs/name, calls model operations, returns stable JSON/status codes.
backend/src/controllers/uploadController.js	Uses Exercise.findByPk and instance.update; removes newly uploaded files on failure.
backend/src/middleware/uploadMiddleware.js	Creates UUID filenames in uploads; accepts allowed extension/MIME pairs; enforces one file and 5 MiB.
backend/uploads/	Disk files served through the /uploads static URL.
backend/BACKEND_LEARNING_GUIDE.md	An existing text guide; the PDF expands the source-grounded explanation and presentation material.

Not found: No backend SQL schema/migration directory or .env.example is present. The root .env.example and README are frontend-oriented; the README still says there is no backend/database, so that statement is outdated relative to the current source.

CHAPTER 04

Dependencies and environment configuration

Installed and locked dependencies

Package	Locked version	Use
express	5.3.0	HTTP server, routing, middleware pipeline.
sequelize	6.37.8	ORM and model operations.
mysql2	3.24.6	Driver used by Sequelize's mysql dialect.
cors	2.8.6	CORS response middleware.
dotenv	18.0.9	Environment-variable loading.
helmet	8.3.0	HTTP security headers.
multer	2.5.0	Multipart upload handling.
nodemon	3.1.14	Development restart tool.

Root frontend dependency versions checked from its lockfile include React 19.3.1, Zustand 5.0.16, and Vite 8.3.5. The frontend build also uses React Router.

Environment variable names (values intentionally withheld)

Backend variable	Purpose in code
DB_HOST	Database host; fallback localhost.
DB_PORT	Database port; fallback 3306.
DB_USER	Database username; fallback root.
DB_PASSWORD	Database password; fallback empty string.
DB_NAME	Database name; fallback fitness_explorer.
PORT	Express listening port; fallback 5000.
CLIENT_ORIGIN	Single allowed CORS origin; fallback http://localhost:5173.
VITE_DATA_SOURCE	Frontend source selector: mock, live, or backend.
VITE_BACKEND_API_URL	Frontend API base; fallback http://localhost:5000/api.

The actual root .env additionally contains VITE_USE_MOCK_DATA, VITE_MOCK_SIMULATE_ERROR and VITE_RAPIDAPI_* names. The current config.js selects by VITE_DATA_SOURCE; VITE_USE_MOCK_DATA is not read there. No .env values or credentials are displayed.

Sequelize connection configuration

```
const sequelize = new Sequelize(
  process.env.DB_NAME || "fitness_explorer",
  process.env.DB_USER || "root",
  process.env.DB_PASSWORD || "",
  {
    dialect: "mysql",
    host: process.env.DB_HOST || "localhost",
    port: Number(process.env.DB_PORT) || 3306,
    logging: false,
    pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
  }
);
await sequelize.authenticate();
```

The single Sequelize instance owns a connection pool. authenticate() checks connectivity and credentials only. Startup does not invoke sync(), force, alter, or migration code.

CHAPTER 05

Database table and Exercise model

Confirmed existing schema

The table details below are supplied by the student and were not retrieved by a database query during PDF generation. The database is fitness_explorer, the existing table is exercises, and the student confirms it contains six exercise records.

Column	Confirmed type / rules	Model mapping
id	INT, primary key, auto-increment	INTEGER, primaryKey, autoIncrement
external_id	VARCHAR(100), nullable, UNIQUE	STRING(100), allowNull, unique
name	VARCHAR(150), NOT NULL	STRING(150), allowNull:false
body_part	VARCHAR(100), nullable, indexed	STRING(100), nullable; index metadata
target_muscle	VARCHAR(100), nullable, indexed	STRING(100), nullable; index metadata
equipment	VARCHAR(100), nullable, indexed	STRING(100), nullable; index metadata
instructions	TEXT, nullable	TEXT, nullable
gif_url	VARCHAR(500), nullable	STRING(500), nullable
created_at	TIMESTAMP, current timestamp	Sequelize createdAt mapped to created_at
updated_at	TIMESTAMP, auto-update current timestamp	Sequelize updatedAt mapped to updated_at

Definition vs operations

`Exercise.js` defines what a row/table looks like to Sequelize: attributes, types, key/unique/index metadata, tableName, and timestamp mappings.

`exerciseModel.js` defines reusable application operations such as list, search, count, fetch one, insert, update, and delete. It imports the Exercise model and calls methods like findAll(), findByPk(), create(), update(), destroy(), and count().

```
const Exercise = sequelize.define("Exercise", attributes, {
  tableName: "exercises",
  timestamps: true,
  createdAt: "created_at",
  updatedAt: "updated_at",
});
```

Sequelize normally has timestamp names createdAt and updatedAt. The explicit mappings make it use the existing snake_case columns. Defining a model alone does not run DDL; the application has no sync call.

> **Schema caution:** There is no schema SQL file in the project. The model metadata is designed to match the confirmed existing table. Do not use automatic alter/force synchronization against the six existing records.

CHAPTER 06

Server startup, middleware, and errors

Express registration order

Order	Registration	Effect
1	Helmet	Adds security headers; CORP is configured as cross-origin for exercise media.
2	CORS	Allows the configured CLIENT_ORIGIN.
3	express.json()	Parses JSON request bodies (Express's default parser limit applies; no custom limit is set).
4	express.static('/uploads')	Serves image files from backend/uploads.
5	app.use('/api/exercises', exerciseRoutes)	Mounts CRUD and upload route definitions.
6	GET /api/health	Returns a basic success message and timestamp; does not query DB on each call.
7	Error middleware	Maps Multer size errors to 413, other Multer/type errors to 400, remaining errors to 500.

Startup sequence

```
async function startServer() {
  try {
    await testDatabaseConnection(); // Sequelize authenticate()
    app.listen(PORT, () => {
      console.log(`Fitness Explorer API running on http://localhost:${PORT}`);
    });
  } catch (error) {
    console.error("Server startup failed because the database is unavailable.");
    process.exit(1);
  }
}
startServer();
```

If authentication fails, the server does not begin listening. The catch logs a generic startup message and exits. The server does not implement a retry loop or a graceful pool shutdown handler.

Error response behavior

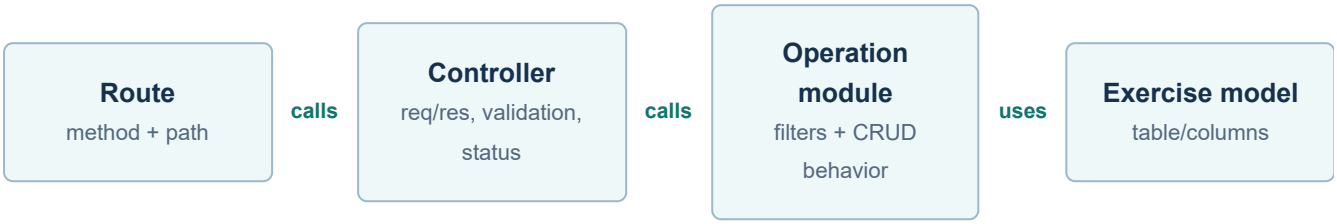
- Exercise controllers catch operation failures, log the operation-specific error message, and usually send a generic 500 response.
- Invalid identifiers and missing exercise names return 400; absent records return 404.
- SequelizeUniqueConstraintError is mapped to 409 for duplicate external IDs.
- Upload errors are handled both inside uploadController.js (database/record failures) and in server.js (Multer parsing/type/size failures).

- The health endpoint indicates the HTTP server handler responds; it is not ongoing proof that MariaDB remains available.

CHAPTER 07

Routes, controllers, and model operations

Separation of responsibilities



Routes should remain small: they connect a URL and HTTP method to a controller. Controllers understand Express request/response objects. Model operations understand exercise data and Sequelize. This separation means database behavior can be reused without duplicating HTTP handling.

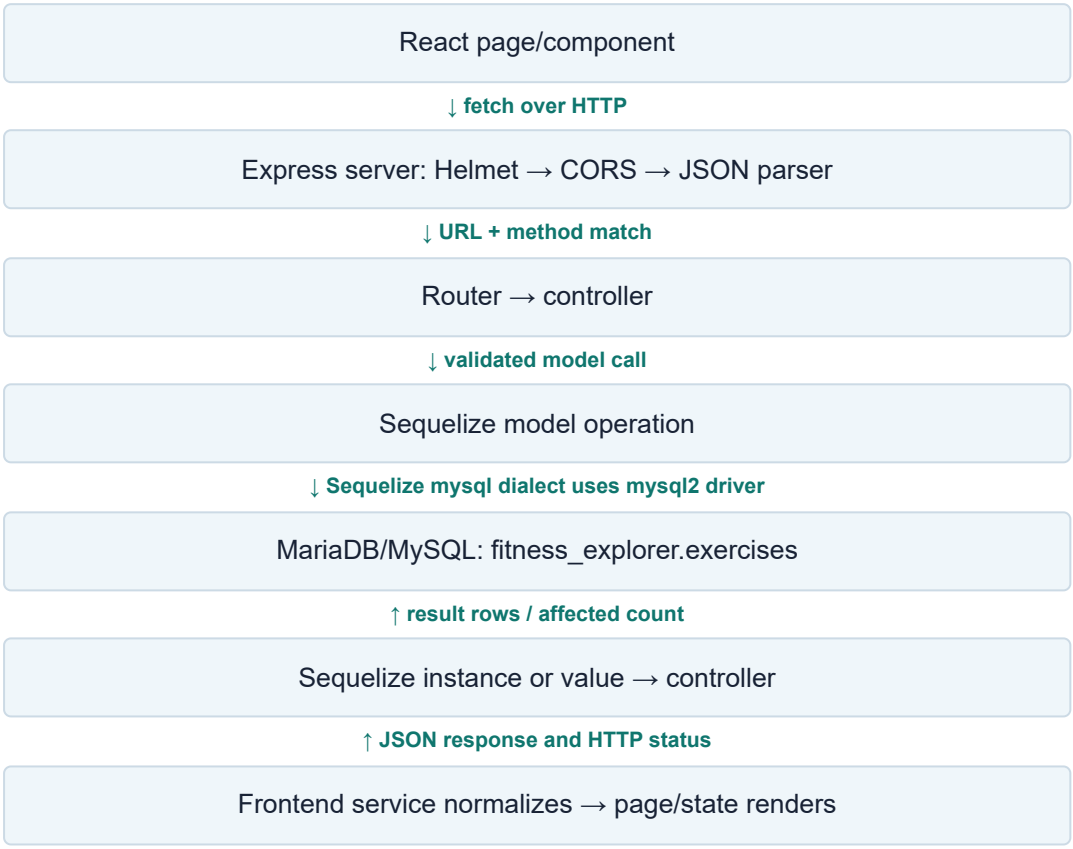
Actual operation mapping

Endpoint	Route handler	Controller	Model operation(s)
GET /api/exercises	router.get('/', getExercises)	getExercises	getAllExercises + countExercises
GET /api/exercises/:id	router.get('/:id', getExercise)	getExercise	getExerciseById → findByPk
POST /api/exercises	router.post('/', addExercise)	addExercise	createExercise → create
PUT /api/exercises/:id	router.put('/:id', editExercise)	editExercise	updateExercise → update
DELETE /api/exercises/:id	router.delete('/:id', removeExercise)	removeExercise	deleteExercise → destroy
POST /api/exercises/:id/upload	upload.single('image'), uploadExerciseImage	uploadExerciseImage	Exercise.findByPk + instance.update

CHAPTER 08

Complete request and response flow

Architecture diagram



Example: GET /api/exercises/2

1. The details page calls `getExerciseById(id)` from the shared frontend service.
2. In backend mode, `backendExerciseApi.js` fetches `/exercises/2` at the configured backend API base.
3. Express middleware runs, then `exerciseRoutes.js` matches `GET /:id` and selects `getExercise`.
4. The controller converts the path parameter to a number and rejects non-positive/non-integer IDs with 400.
5. It calls `getExerciseById(id)` in the model operations module, which calls `Exercise.findByPk(id)`.
6. Sequelize translates the model request into a database lookup via mysql2. If no row is found, the controller sends 404; otherwise it returns 200.
7. The frontend validates the response envelope and normalizes database field names for the UI.

```
// Illustrative response shape; not a captured database dump:
{
  "success": true,
  "data": {
    "id": 2,
    "name": "Push Up",
    "body_part": "…",
    "target_muscle": "…",
    "equipment": "…",
    "instructions": "…",
    "gif_url": "…"
  }
}
```

The student-provided manual test says ID 2 returned the Push Up record correctly. The exact field values are not shown here because they were not supplied as test output.

CHAPTER 09

API endpoint reference

Method and path	Purpose / inputs	Success response and common errors
GET /api/health	Health message; no parameters.	200: success, message, timestamp. It does not check DB on every call.
GET /api/exercises	Query: search, body_part, target_muscle, equipment, page, limit, sortBy, order.	200: success, pagination, count, data. 500 on query failure.
POST /api/exercises	JSON exercise fields; name is required and trimmed.	201: success, message, data with generated id. 400 invalid name; 409 duplicate external_id; 500 other error.
GET /api/exercises/:id	Positive integer id in path.	200: success, data. 400 invalid ID; 404 not found; 500 other failure.
PUT /api/exercises/:id	Positive integer id; JSON requires name and accepts editable fields.	200: success, message. 400 invalid ID/name; 404 zero affected; 409 duplicate ID; 500 other failure.
DELETE /api/exercises/:id	Positive integer id.	200: success, message. 400 invalid ID; 404 no row deleted; 500 failure.
POST /api/exercises/:id/upload	Multipart file field exactly image; positive exercise id.	200: success, message, data.exerciseId/imageUrl/filename. 400 invalid/no file/type; 404 missing exercise; 413 >5 MiB; 500 DB failure.
GET /uploads/:filename	Static file path, not JSON API route.	Image/file bytes from backend/uploads; normal static-file 404 if missing.

Response envelope examples

```
// List response
{ "success": true,
  "pagination": { "currentPage": 1, "limit": 10, "totalExercises": 6,
    "totalPages": 1, "hasNextPage": false, "hasPreviousPage": false },
  "count": 6, "data": [ /* Sequelize records */ ] }

// Upload response
{ "success": true, "message": "Exercise image uploaded successfully!",
  "data": { "exerciseId": 2, "imageUrl": "/uploads/<uuid>.jpg",
            "filename": "<uuid>.jpg" } }
```

The list's numeric example reflects the six records you confirmed and is illustrative; this task did not make an API request.

CHAPTER 10

CRUD explained with Sequelize

Create — POST /api/exercises

Flow: route → `addExercise` → `createExercise` → `Exercise.create(values)`. The controller requires a non-empty string name, trims it, packages accepted fields, and returns 201 with the generated ID.

```
const createdExercise = await Exercise.create({
  external_id: exercise.external_id || null,
  name: exercise.name,
  body_part: exercise.body_part || null,
  target_muscle: exercise.target_muscle || null,
  equipment: exercise.equipment || null,
  instructions: exercise.instructions || null,
  gif_url: exercise.gif_url || null,
});
return createdExercise.id;
```

```
// Example request
POST /api/exercises
Content-Type: application/json
{
  "external_id": "walkthrough-demo-unique",
  "name": "Walkthrough Test Exercise",
  "body_part": "upper legs",
  "target_muscle": "quadriceps",
  "equipment": "body weight",
  "instructions": "Temporary demonstration record",
  "gif_url": null
}

// Response shape
HTTP 201
{ "success": true, "message": "Exercise created successfully.",
  "data": { "id": 123, "external_id": "walkthrough-demo-unique",
            "name": "Walkthrough Test Exercise", ... } }
```

The response ID above is a placeholder, not a real ID. Sequelize inserts the row using the model; the MariaDB auto-increment column supplies the primary key.

Read — GET /api/exercises and GET /api/exercises/:id

Read type	Controller/model path	Sequelize behavior
All / filtered	getExercises → getAllExercises	findAll({ where, order, limit, offset }); count() obtains matching total.
Single row	getExercise → getExerciseById	findByPk(id) returns an instance or null.

Update — PUT /api/exercises/:id

The controller validates the positive ID and non-empty name, then calls `updateExercise`. That function calls static `Exercise.update(values, { where: { id } })` and returns affected row count. The current PUT

replaces all listed fields; omitted optional fields become null. Zero affected rows are interpreted as 404, although some database configurations may report zero for unchanged values.

```
PUT /api/exercises/123
Content-Type: application/json
{
  "external_id": "walkthrough-demo-unique",
  "name": "Walkthrough Test Exercise Updated",
  "body_part": "upper legs",
  "target_muscle": "glutes",
  "equipment": "body weight",
  "instructions": "Updated temporary demonstration record",
  "gif_url": null
}
// { "success": true, "message": "Exercise updated successfully." }
```

Delete — DELETE /api/exercises/:id

`deleteExercise` calls `Exercise.destroy({ where: { id } })`. The controller maps zero deletions to 404 and otherwise returns 200.

```
DELETE /api/exercises/123
// { "success": true, "message": "Exercise deleted successfully." }
```

> **Safe demonstration rule:** only delete the ID returned by a newly created, clearly named test record. Never substitute one of the six original exercise IDs.

CHAPTER 11

Search, filtering, sorting, and pagination

Filter construction

```
const searchValue = `%${filters.search}%`;
conditions.push({
  [Op.or]: [
    { name: { [Op.like]: searchValue } },
    { body_part: { [Op.like]: searchValue } },
    { target_muscle: { [Op.like]: searchValue } },
    { equipment: { [Op.like]: searchValue } },
  ],
});
return conditions.length ? { [Op.and]: conditions } : {};
```

Query parameter	Behavior
search	LIKE pattern across name/body_part/target_muscle/equipment; wildcard behavior for % and _ follows the database collation/LIKE semantics.
body_part	Exact match.
target_muscle	Exact match.
equipment	Exact match.
sortBy	Allowlist: id, name, body_part, target_muscle, equipment, created_at; invalid value falls back to id.
order	ASC if case-insensitively equal to ASC; otherwise DESC.
page	Parsed integer, minimum 1, default 1.
limit	Parsed integer, default 10, clamped to 1 through 100.

Sequelize list options

```
Exercise.findAll({
  where,
  order: [[safeSortBy, safeOrder]],
  limit: pagination.limit,
  offset: pagination.offset,
});
```

`where` holds the filters. `Op.like`, `Op.or`, and `Op.and` express matching rules. `order` supplies a validated field and direction. `limit` caps result rows; `offset` skips the rows from earlier pages. The current source does not use a custom `attributes` selection, so Sequelize returns the model attributes.

Offset formula: `(page - 1) × limit`. For page 2 and limit 10, the query skips 10 matching rows.

```
GET /api/exercises?search=push&page=1&limit=5&sortBy=name&order=ASC
GET /api/exercises?body_part=chest&target_muscle=pectorals
GET /api/exercises?equipment=dumbbell
```

The controller calls Exercise.count() through countExercises() with the same filter conditions to calculate totalExercises and totalPages. It returns hasNextPage and hasPreviousPage booleans. The frontend backend client currently returns only normalized data rows, so Explore applies its own in-memory display sorting and 'Show more' behavior on the returned set.

CHAPTER 12

Image upload from browser to disk and database

End-to-end sequence

1. The user opens an existing backend exercise detail page and selects one image.
2. `ExerciseImageUpload.jsx` checks the filename extension, MIME type when provided, nonzero size, and 5 MiB maximum. It shows a local object-URL preview, filename, and size; selection alone does not upload.
3. On confirmation, the component creates FormData and appends the file under the exact key `image`.
4. The frontend service sends POST to `/api/exercises/:id/upload` without manually specifying multipart Content-Type, allowing the browser to attach its boundary.
5. Express routes the request through Multer's `upload.single("image")`.
6. Multer validates extension/MIME, creates a UUID-based filename, and writes the file in `backend/uploads/`.
7. The controller checks the ID, finds the exercise with `Exercise.findByPk()`, and calls `exercise.update({ gif_url: imageUrl })`.
8. The controller responds with a relative URL and filename.
9. React resolves the relative URL against the configured backend API base, updates the details view immediately, and synchronizes matching image references in favorites, recent exercises, and workout state.
10. Express static middleware serves the file at `/uploads/<filename>`.

Storage and controls

Control	Actual implementation
Storage	Multer diskStorage; backend/uploads folder.
Filename	crypto.randomUUID() + lowercased original extension.
Types	jpg/jpeg + image/jpeg; png + image/png; gif + image/gif; webp + image/webp.
Count and size	One file per request; 5 * 1024 * 1024 byte maximum.
Database field	gif_url stores a relative /uploads/. path.
File URL	Express static route /uploads maps to the upload directory.
On missing exercise / invalid ID / DB error	Controller attempts to unlink the just-uploaded file; non-ENOENT cleanup failures are logged.

Practical upload example

```
curl.exe -sS -X POST `
  -F "image=@C:\path\to\test-image.jpg" `
  "http://localhost:5000/api/exercises/123/upload"
```

> **Do not use ID 123 blindly.** In the example, use the ID of a temporary test exercise you created, not an original row.

Limitations found in the source

- On successful replacement, the old image file is not deleted. It can become an unreferenced file on disk.
- Deleting an exercise does not delete its image file.
- Multer validates the client-provided extension and MIME label, not decoded image contents or file signatures; both labels can be spoofed.
- If Multer rejects the upload before the controller runs, cleanup behavior for a partially written file depends on Multer's handling.
- Frontend allows image/jpg for .jpg, but backend accepts only image/jpeg, so that uncommon MIME label may be rejected.
- Disk storage is simple for a local project, but multiple app instances or ephemeral hosting need shared/persistent storage.

CHAPTER 13

React, API client, and Zustand integration

Data-source switch

`src/services/config.js` reads `VITE_DATA_SOURCE` and defaults to mock. `src/services/exerciseApi.js` chooses mock, live ExerciseDB, or the backend API module. Uploads are rejected unless backend mode is selected. Demo IDs beginning with `demo-` are served from the local mock dataset.

Backend request and response normalization

Database/API property	Frontend property
id	String(raw.id)
body_part	bodyPart
target_muscle	target
gif_url	gifUrl (resolved to absolute backend URL)
instructions string	instructions array containing that string
equipment	equipment

The backend service uses fetch(), checks HTTP status and JSON shape, and throws ApiRequestError or NotFoundError for failed operations. Upload uses FormData and expects a successful response with data.imageUrl.

Pages and components using the shared service

- ExplorePage.jsx calls getExercises() with search and filter values; useAsyncData provides loading, error, and retry behavior.
- HomePage.jsx calls getExercises() for the dashboard's exercise data.
- ExerciseDetailsPage.jsx calls getExerciseById(), remembers the exercise through the recent Zustand store, and mounts the uploader for backend records.
- RelatedExercises.jsx calls the shared exercise service for its related list when applicable.
- ExerciseImageUpload.jsx updates favorites, recent, workout state, and the currently displayed details image after success.

Zustand responsibilities

Zustand is frontend state management; it does not replace Express or MariaDB. The favorites, recent, and workout stores persist selected app state in localStorage. The upload component invokes store actions to update copied exercise objects after the server returns the new image URL.

Documentation mismatch observed

The root README and root .env.example still describe a frontend-only project and VITE_USE_MOCK_DATA, while the actual current selector uses VITE_DATA_SOURCE and the backend uses a separate backend/.env. This report does not modify those files.

CHAPTER 14

PowerShell and curl.exe practice commands

> **Safety first:** Create a temporary exercise using the POST command, capture its returned ID, and use only that ID for PUT/DELETE/upload practice. Never put an original exercise ID in the DELETE example. Replace the unique external_id if re-running CREATE after a previous record remains.

PowerShell: health, create, read, update, search/filter/sort/page

```
$Api = 'http://localhost:5000/api'

# 1. Health (read-only)
Invoke-RestMethod "$Api/health"

# 2. Create a disposable test row. Save the returned ID.
$payload = @{
  external_id = "walkthrough-$([guid]::NewGuid())"
  name = "Walkthrough Test Exercise"
  body_part = "upper legs"
  target_muscle = "quadriceps"
  equipment = "body weight"
  instructions = "Temporary API practice record"
  gif_url = $null
} | ConvertTo-Json
$created = Invoke-RestMethod "$Api/exercises" -Method Post `
  -ContentType 'application/json' -Body $payload
$testId = $created.data.id
"Created temporary exercise ID: $testId"

# 3. List records; inspect $result.pagination and $result.data.
$result = Invoke-RestMethod "$Api/exercises?page=1&limit=10"

# 4. Read the temporary row (not an original row).
Invoke-RestMethod "$Api/exercises/$testId"

# 5. Update only the temporary test row. PUT requires name.
$update = @{
  external_id = $created.data.external_id
  name = "Walkthrough Test Exercise Updated"
  body_part = "upper legs"
  target_muscle = "glutes"
  equipment = "body weight"
  instructions = "Updated temporary test data"
  gif_url = $null
} | ConvertTo-Json
Invoke-RestMethod "$Api/exercises/$testId" -Method Put `
  -ContentType 'application/json' -Body $update

# 6. Search and combine exact filters
Invoke-RestMethod "$Api/exercises?search=walkthrough"
Invoke-RestMethod "$Api/exercises?body_part=upper%20legs&equipment=body%20weight"

# 7. Sort and paginate
Invoke-RestMethod "$Api/exercises?sortBy=name&order=ASC&page=1&limit=5"

# 8. Upload to the temporary exercise only (PowerShell curl.exe)
curl.exe -sS -X POST -F "image=@C:\path\to\test-image.jpg" `
  "$Api/exercises/$testId/upload"

# 9. Delete only the temporary test row you created above.
# Verify $testId is the returned temporary ID before running this line.
Invoke-RestMethod "$Api/exercises/$testId" -Method Delete
```

curl.exe: matching requests

```
# Health
curl.exe -sS "http://localhost:5000/api/health"

# Create; use a fresh unique external_id. Copy the returned id as TEST_ID.
curl.exe -sS -X POST "http://localhost:5000/api/exercises" `
  -H "Content-Type: application/json" `
  -d '{"external_id":"walkthrough-UNIQUE-STRING","name":"Walkthrough Test Exercise","body_part":"upper
legs","target_muscle":"quadriceps","equipment":"body weight","instructions":"Temporary API practice
record","gif_url":null}'

# List, search, filters, sorting, pagination
curl.exe -sS "http://localhost:5000/api/exercises?page=1&limit=10"
curl.exe -sS "http://localhost:5000/api/exercises?search=walkthrough"
curl.exe -sS "http://localhost:5000/api/exercises?body_part=upper%20legs&equipment=body%20weight"
curl.exe -sS "http://localhost:5000/api/exercises?sortBy=name&order=ASC&page=1&limit=5"

# Read temporary row
curl.exe -sS "http://localhost:5000/api/exercises/TEST_ID"

# Update temporary row; replace TEST_ID and values as appropriate
curl.exe -sS -X PUT "http://localhost:5000/api/exercises/TEST_ID" `
  -H "Content-Type: application/json" `
  -d '{"external_id":"walkthrough-UNIQUE-STRING","name":"Walkthrough Test Exercise
Updated","body_part":"upper legs","target_muscle":"glutes","equipment":"body weight","instructions":"Updated
temporary test data","gif_url":null}'

# Upload image to temporary row; replace TEST_ID and local file path
curl.exe -sS -X POST -F "image=@C:\path\to\test-image.jpg" `
  "http://localhost:5000/api/exercises/TEST_ID/upload"

# Destructive demonstration: only after verifying TEST_ID belongs to your
# temporary row, never an original exercise.
curl.exe -sS -X DELETE "http://localhost:5000/api/exercises/TEST_ID"
```

PowerShell commands use `Invoke-RestMethod` for JSON and curl.exe for multipart upload. The curl.exe JSON uses a placeholder unique external ID. These commands are examples and were not executed as part of this PDF task.

CHAPTER 15

Testing and verified results

Tests run while generating this PDF

Check	Result in this task
npm run lint	Passed.
npm run build	Passed; Vite production build completed.
node --test	Passed: 9 tests, 0 failures. These are frontend exercise-detail/workout-progress tests, not backend integration tests.
Backend database CRUD	Not run in this task; no DB connection or mutation was made.
Manual PDF-generation examples	Not run against the API.

Manual results supplied by the student

> **Attribution:** The results below were provided in the task prompt as confirmed prior tests. They were not independently rerun for this PDF.

Student-reported test	Reported result
Backend startup	npm run dev; Express on port 5000; Sequelize connected; console reported Database connected successfully!
GET /api/health	success true and expected API-running message.
GET /api/exercises	success true; six original rows returned.
GET /api/exercises/2	Push Up record returned correctly.
CREATE	Temporary Sequelize CRUD Test Exercise received ID 8; success response.
UPDATE	Temporary row renamed and target muscle/instructions updated; subsequent GET confirmed changes.
DELETE	Temporary ID 8 deleted; success response.
Final check	Six original rows remained; temporary row gone; original image paths preserved.

Automated backend test status

No backend automated test files were found. The backend package's test script is a placeholder that prints an error and exits 1. The existing nine Node tests are frontend unit tests. Do not describe the supplied manual CRUD evidence as an automated test suite.

Remaining verification

- Repeat API CRUD and upload testing against a safe development database, verifying no original row is changed.
- Test invalid IDs, duplicate external_id, missing name, empty upload, invalid MIME/extension, and files over 5 MiB.
- Test the frontend from its actual configured origin against the backend CORS setting.
- Add backend tests with a separate test database or mocked Sequelize model before relying on regression coverage.

CHAPTER 16

Security, limitations, and future improvements

Area	Present behavior	Risk / reason to improve
Authentication /authorization	No auth middleware appears in routes/server.	Anyone who can reach the API can attempt create/update/delete/upload; add identity and role checks before public deployment.
SQL injection	Sequelize model operations and Op operators; sort field allowlist.	Values are not concatenated into application-written SQL; still validate input and keep Sequelize/current dependencies maintained.
Upload validation	Extension and MIME allowlist, one file, 5 MiB.	MIME/extension are client-controlled; verify actual file signatures/decode content, and consider image re-encoding.
Upload cleanup	New file removed on invalid ID, missing exercise, or caught DB failure.	Old file on successful replacement and image on exercise deletion are not removed.
Request size	Multer per-file 5 MiB; express.json() uses parser default because no custom limit is set.	Define deliberate body limits and handle malformed/oversized request bodies clearly.
Rate limiting	No rate limiter configured.	Abuse can consume CPU, DB, and disk; add rate and upload quotas.
Errors	Controllers return generic 500 messages; global handler handles Multer/type errors.	Good not to expose internal details, but production should use structured logging and consistent request IDs.
DB startup	sequelize.authenticate() before listen; pool max 10.	No retry/readiness monitoring or graceful shutdown; health endpoint does not test DB after boot.
Environment	dotenv with DB values; VITE_ settings are browser-visible.	Never put DB secrets in root VITE_ variables; ensure .env files are ignored and deployment secrets are managed externally.
CORS	One configured CLIENT_ORIGIN.	Exact origin configuration is needed in deployment; CORS is not authentication.
Schema management	No migrations or sync calls.	Safe for existing data, but schema changes need reviewed migrations and backups.
Input validation	Positive IDs, required non-empty name, pagination clamps, sort allowlist.	Field types/lengths and query parameter shapes are not comprehensively validated.
Pagination	limit <=100; page min 1.	Very large page can produce a large offset and inefficient query.
Update semantics	PUT writes nullable fields as null if absent.	Clients must send full data; consider explicit PATCH semantics if partial updates are desired.
Unique errors	SequelizeUniqueConstraintError mapped to 409.	Verify error mapping with the deployed driver/database in an integration test.
Docs	Backend guide exists; root README/.env.example remain frontend-only.	New developers may miss backend setup or choose the wrong data-source variable.
Automated backend tests	None found; npm test is placeholder.	CRUD/upload behavior can regress without backend-focused tests.

Improvements that preserve safety

- Add a test suite isolated from the real six-record database; never use destructive sync options.
- Add authentication/authorization, rate limits, explicit body limits, and stricter request validation before deployment.
- Validate image content, define old-file retention/deletion policy, and store uploads on persistent/shared storage in multi-instance deployment.
- Add structured logs, readiness checks, graceful shutdown, and startup retry/operational monitoring.
- Document backend setup and variable names in a backend-specific example without secret values.
- Use reviewed schema migrations for future changes; keep force/alter sync disabled.

CHAPTER 17

Supervisor presentation walkthrough

Suggested order for a short demonstration. Use a development environment and a temporary record. Do not show credentials or delete an original row.

Step	What to show / do	Natural speaking note
1. Introduce project	Open Fitness Explorer in browser.	“This project helps users discover exercises and organize training.”
2. Why backend	Show exercise details and database table in a safe DB viewer.	“The backend centralizes exercise records and image URLs instead of requiring a browser-to-database connection.”
3. Folder tree	Open backend/src folder.	“Routes direct requests, controllers handle HTTP, models handle data, middleware handles uploads.”
4. Technologies	Open backend/package.json.	“Node runs the server; Express handles HTTP; Sequelize models rows; mysql2 speaks the DB protocol.”
5. Sequelize vs mysql2	Open package entries and config/db.js.	“Sequelize is the ORM; mysql2 is the driver underneath; MariaDB stores the data.”
6. DB connection	Open config/db.js without showing .env values.	“Environment variables supply credentials; authenticate checks connectivity.”
7. Server setup	Open server.js.	“Middleware runs before routes; server listens only after DB authentication.”
8. Routes/controllers/models	Open exerciseRoutes.js, controller, Exercise.js, exerciseModel.js.	“Route selects controller; controller validates and formats response; operation module uses the Exercise model.”
9. CRUD	Use API client with a new temporary record.	“Create a disposable row, read and update it, then delete only that row.”
10. Search and filters	Send GET with query parameters.	“Op.like searches fields; exact filters combine; sort is allowlisted; limit/offset paginate.”
11. Upload	Use temporary exercise and a test image.	“Multer saves a UUID file; Sequelize stores its relative path in gif_url.”
12. React/Zustand	Open backendExerciseApi.js and upload component/store actions.	“The service normalizes data; Zustand stores frontend copies, not backend database rows.”
13. API demo	Show health/list/detail endpoints.	“The response is JSON; the frontend checks success/data before rendering.”
14. Tests/limits	Show test output and report limitations.	“Frontend tests exist; backend CRUD tests were manual per my reported results; this PDF run did not mutate the DB.”
15. Conclusion	Summarize architecture diagram.	“The API now connects React to MariaDB through Express, Sequelize, and mysql2 while preserving the existing table.”

Demonstration sequence

1. Show the React details page and describe the user-facing exercise library.
2. Show backend folders, then package dependencies.
3. Open config/db.js and Exercise.js; explain no schema sync is used.

4. Open routes and controller, then show the matching Sequelize operation.
5. Call health, list, and detail endpoints.
6. Create a uniquely named temporary record; save its returned ID; read and update it.
7. Use it to upload a test image; show the relative DB path and browser image URL.
8. Delete only the temporary row and verify the original six remain.
9. Finish with tests/known limitations and future improvements.

In the current task, no live demonstration requests were made. The student-provided prior test report says a temporary ID 8 was created, updated, deleted, and the final total returned to six.

CHAPTER 18

Supervisor questions and answers

1. What is a backend?

The server-side part that handles API requests, validation, shared data operations, and communication with the database.

2. Why Node.js?

The project uses JavaScript on the server, so Node.js runs the backend JavaScript runtime.

3. What is Express?

A Node.js web framework used here to register middleware, routes, and HTTP handlers.

4. What is an API?

A defined way for the React client to request backend data or operations over HTTP.

5. What is REST?

A common API style using HTTP methods and resource paths; this project uses GET, POST, PUT, and DELETE for exercises.

6. What is CRUD?

Create, Read, Update, and Delete—the four basic data operations implemented for exercises.

7. What is Sequelize?

An ORM that maps JavaScript model calls to database operations.

8. Why Sequelize instead of application-written raw SQL?

The supervisor required ORM use, and model methods centralize data operations in JavaScript.

9. What is an ORM?

A layer that represents database tables as models and rows as objects.

10. What is mysql2?

The MySQL-protocol driver used beneath Sequelize's mysql dialect.

11. How are Sequelize and mysql2 different?

Sequelize is the higher-level ORM; mysql2 handles the database protocol connection underneath.

12. What is a database connection?

A communication session between the server process and the database service.

13. What is a connection pool?

A reusable set of database connections managed by Sequelize; the configured maximum is 10.

14. What is a Sequelize model?

A definition of a table's attributes and metadata; Exercise maps to the existing exercises table.

15. What is the purpose of routes?

They map an HTTP method and URL to middleware/controller functions.

16. Routes vs controllers vs models?

Routes match URLs; controllers handle HTTP inputs/responses; model operations interact with Exercise records.

17. What is middleware?

Code that runs in the request pipeline before a final route handler, such as CORS or Multer.

18. What is Multer?

Express middleware used here to parse multipart form data and store one file on disk.

19. How does image upload work?

React sends FormData; Multer stores a UUID file; the controller updates gif_url using the Exercise instance.

20. What is JSON?

A text data format used for API request/response bodies.

21. What is CORS?

Browser cross-origin permission headers; the server allows the configured client origin.

22. How does React communicate with Express?

The backend service uses fetch() against VITE_BACKEND_API_URL.

23. Why Zustand?

It manages frontend state such as favorites, recent items, and workout plans; it is separate from backend persistence.

24. How does pagination work?

Page and limit determine offset; findAll returns a slice; count provides total metadata.

25. How does Sequelize reduce injection risk?

Model operations bind values; the code also allowlists sort columns. Validation is still necessary.

26. What if MariaDB is unavailable at startup?

authenticate fails, server logs startup failure, and the process exits instead of listening.

27. Why environment variables?

They keep deployment settings and DB credentials outside source code.

28. Why store images on disk?

Multer's diskStorage writes files to the uploads folder; the database stores a URL path, not image bytes.

29. What are timestamps for?

created_at records row creation; updated_at indicates last update time.

30. Why avoid automatic synchronization?

It could change or recreate an existing schema; this project must preserve the existing table and records.

31. What is the Exercise table's primary key?

The confirmed id integer column, auto-incremented.

32. What does external_id unique mean?

Non-null external identifiers cannot be duplicated; the database constraint produces a conflict mapped to HTTP 409.

33. What does findByPk return when missing?

It returns null, which the controller maps to a 404 response.

34. How does search combine with filters?

Search fields are ORed; search and exact filters are combined with AND.

35. Why is sortBy allowlisted?

Only known database fields should be used for ordering; untrusted column names are rejected to a safe default.

36. What is FormData?

A browser API for multipart form requests, suitable for file uploads.

37. Why not set multipart Content-Type manually?

The browser must add the correct multipart boundary along with the header.

38. Does replacing an image remove the old one?

No. The current code cleans a newly uploaded file on error, but does not delete an old file after success.

39. Are backend requests authenticated?

No authentication or authorization middleware is present in the current routes.

40. Does health check verify MariaDB each time?

No. It returns a server response and timestamp; database is checked at startup.

41. What automated backend tests exist?

None were found; backend npm test is a placeholder. The nine current tests are frontend unit tests.

42. What is the frontend data selector?

VITE_DATA_SOURCE, with mock as the default and backend/live as alternatives.

43. What does frontend normalization do?

It converts DB snake_case fields into names the React components expect and resolves image URLs.

44. What status code means created?

201 Created for POST /api/exercises.

45. What status code means no row found?

404 Not Found for a missing exercise.

46. What would improve the next version?

Add backend integration tests, authentication, rate limiting, stricter image validation, structured logging, and migration tooling that is reviewed and non-destructive.

CHAPTER 19

Student-level conclusion

Fitness Explorer combines an existing React interface with a backend that exposes exercise and image-upload functionality through Express routes. The backend separates URL matching, HTTP validation, and database operations into routes, controllers, and Sequelize model layers.

Sequelize provides the ORM operations for the existing MariaDB/MySQL exercises table, while mysql2 is the underlying driver. The Exercise model explicitly maps the existing table name, columns, and timestamps. Startup authenticates to the database and does not synchronize or recreate the schema.

The frontend communicates through a service module that selects mock, live, or backend data. It normalizes database fields to the React component shape. Image upload uses FormData and Multer disk storage; the database keeps a relative URL, and Express serves the file from the uploads directory. Zustand remains responsible for selected frontend state.

The student-reported manual tests demonstrate startup, health, listing, reading one exercise, and a temporary create/update/delete cycle that ended with the six original exercises still present. In this PDF task, I independently ran lint, the frontend build, and the existing nine frontend tests, but did not connect to or alter MariaDB.

Current limitations include no backend automated test suite, no authentication, limited input/file-content verification, no old-image deletion after replacement, and stale frontend README/environment-example documentation. Future improvements should add isolated integration tests, security controls, clearer operational documentation, persistent upload storage, and reviewed migrations without destructive synchronization.

The main learning outcome is how the browser, HTTP server, ORM, driver, and database each play a different role—and how a request moves through those layers while the existing database remains intact.